OPERATING SYSTEMS • SUSTAINABLE & GREEN COMPUTING

ENERGY-OPTIMAL TASK SCHEDULING USING CARBON-AWARE COMPUTING

Design and Evaluation of an OS-Level Carbon-Aware CPU Scheduling Framework

Authors	Shashwat — 24BCE2409 Lakshyajeet Narsinghani — 24BCE2419
Course	Operating Systems Project
Research Area	OS Process / CPU Scheduling × Sustainable & Green Computing
Current Implementation	Conventional FCFS and Carbon-Aware FCFS
Future System	SJF • SRTF • Priority • Round Robin • DVFS • Virtual-Machine Validation

PROJECT POSITION

This paper documents the first experimental stage of a larger research project. FCFS is the current working scheduler. The architecture is intentionally designed so that additional scheduling policies and virtual-machine validation can be added without changing the overall experimental methodology.

September 2026

RESEARCH SUMMARY

Abstract

Modern computing systems do not consume electricity under a constant environmental condition. The carbon intensity associated with electricity can change throughout the day depending on the electricity-generation mix. As a result, two executions that consume approximately the same amount of electrical energy can have different carbon footprints when they occur at different times. This observation motivates carbon-aware computing, where the timing and location of computation become part of the scheduling problem.

This project studies how the idea of carbon-aware computing can be brought down to the operating-system CPU scheduling level. Instead of treating carbon intensity as a datacenter-level planning parameter, the proposed framework treats it as an additional input available to a CPU scheduler. The first implementation extends First-Come, First-Served (FCFS) scheduling with a temporal carbon-awareness mechanism. When a process reaches the front of the FCFS queue during a high-carbon period, the scheduler checks whether it can safely postpone the process to a lower-carbon period without missing its deadline. If a feasible lower-carbon window exists, execution is shifted; otherwise, the process executes immediately.

The current prototype is implemented as a discrete-event simulation in C. It uses a synthetic time-varying carbon-intensity trace, a constant 50 W CPU power model, a configurable carbon threshold and workload files containing process arrival time, burst time and deadline. Five workload configurations are evaluated using conventional FCFS and carbon-aware FCFS. The experimental results show a consistent reduction in simulated carbon emissions for the carbon-aware version, while waiting time and turnaround time increase because some execution is deliberately postponed. Energy remains unchanged under the constant-power model, which demonstrates an important distinction between energy-aware and carbon-aware scheduling.

The current implementation is not intended to represent the final system. It establishes the algorithmic and experimental foundation for a larger framework that will include SJF, SRTF, Priority Scheduling and Round Robin, followed by richer power and DVFS modeling and validation inside controlled virtual machines. The final objective is to evaluate the carbon benefit together with traditional operating-system performance metrics rather than optimizing carbon in isolation.

Keywords: carbon-aware computing; CPU scheduling; FCFS; operating systems; carbon intensity; temporal load shifting; energy efficiency; sustainable computing; DVFS; virtual-machine validation

Document Overview

The paper is organized so that a reader can understand the project without first reading the source code.

- Sections 1–3 explain the motivation, research background and problem being addressed.
- Sections 4–6 describe the proposed architecture, FCFS extension and carbon-accounting model.
- Sections 7–9 explain the experimental setup and interpret the current results.
- Sections 10–12 discuss limitations, the planned multi-algorithm framework and VM validation.
- Sections 13–15 cover reproducibility, conclusion and references.

1. Introduction

1.1 Motivation

CPU scheduling is one of the fundamental responsibilities of an operating system. The scheduler decides which process receives CPU time and when that process should execute. Traditional scheduling policies are normally designed around objectives such as low waiting time, low turnaround time, fairness,

responsiveness and high CPU utilization. Environmental impact is usually not part of the scheduling decision.

At the same time, computing has become a significant consumer of electricity. A conventional energy-aware scheduler may attempt to reduce the amount of energy consumed by changing CPU frequency, placing tasks on efficient cores or using power-management techniques. However, reducing energy does not automatically mean reducing carbon emissions. The carbon impact of electricity depends on how that electricity is generated at a particular time.

Carbon-aware computing therefore introduces a different question: if a task does not have to execute immediately, can it be moved to a period when the electricity supply is associated with lower carbon intensity? This is the central idea explored by this project.

1.2 From Datacenter Carbon Awareness to OS Scheduling

A large portion of existing carbon-aware scheduling research operates at the level of datacenters, cloud platforms, distributed workloads or geographic regions. Such systems can shift workloads between time periods or locations. The project takes a narrower operating-systems perspective. The goal is to examine whether a similar decision can be made by the CPU scheduler of a single machine.

This distinction is important. A datacenter scheduler may decide where an entire workload should run, while an OS scheduler decides which local process should receive CPU service. The available information, time scale and performance constraints are different. The project therefore treats carbon intensity as a scheduler input alongside familiar process properties such as arrival time, burst time and deadline.

2. Research Background

2.1 Carbon-Aware Computing

Carbon-aware computing considers the carbon intensity associated with electricity when deciding when or where computation should occur. Temporal shifting moves flexible work toward cleaner periods. Spatial shifting moves work toward regions with cleaner electricity. This project concentrates on temporal shifting because it maps naturally to a single-machine CPU scheduling problem.

2.2 Energy-Aware versus Carbon-Aware Scheduling

Energy-aware scheduling and carbon-aware scheduling are related but not identical. Energy-aware scheduling asks how much electricity is consumed. Carbon-aware scheduling asks how much carbon is associated with that electricity. If a processor consumes the same energy at two different times, but the electricity has different carbon intensities, the corresponding carbon emissions will also differ.

KEY CONCEPT

The project deliberately separates energy from carbon. Under the current constant-power model, the same CPU work consumes the same modeled energy, but the estimated carbon changes because the work is executed under different carbon-intensity values.

2.3 Existing OS-Level Energy Awareness

Energy-aware CPU scheduling already exists in real operating systems. The project review examined mechanisms including Linux Energy Aware Scheduling, learning-based extensions to EAS, SCHED_DEADLINE combined with CPU frequency management, and energy-aware process scheduling

approaches. These systems demonstrate that CPU scheduling and power-management decisions can be integrated at the OS level.

The gap motivating this project is the additional use of an external, time-varying electricity carbon-intensity signal. In other words, the operating system may know how much power a CPU is using, but the prototype investigates what happens if the scheduler also knows whether the electricity being consumed is relatively carbon-intensive at that moment.

2.4 Broader Research Gaps

The literature review also highlights several broader challenges. Carbon-aware scheduling must eventually deal with forecast errors, interactive and latency-sensitive workloads, fine-grained task dependencies, network and data-transfer carbon, cooling, fairness, adaptive learning, renewable intermittency and real-world validation. These issues are outside the current FCFS prototype, but they influence the planned architecture.

- Forecast uncertainty can make a predicted clean period different from the actual clean period.
- Interactive applications may not tolerate deliberate waiting in the same way as batch workloads.
- Shared systems raise fairness questions when one process is delayed to reduce system-wide carbon.
- Real hardware introduces power behavior, thermal effects, multicore interactions and scheduler overhead.
- Standardized carbon accounting is required before results from different systems can be compared fairly.

3. Problem Definition and Objectives

3.1 Problem Statement

Consider a set of CPU-bound processes. Each process has an arrival time, CPU burst time and deadline. The machine also has a time-varying carbon-intensity trace. The scheduling problem is to determine the execution time of each process while preserving the base scheduling policy and attempting to reduce the carbon impact of execution. A process may be delayed only when that delay remains feasible with respect to its deadline.

3.2 Research Questions

- Can a classical CPU scheduling policy use carbon intensity as an additional scheduling signal?
- Can computation be shifted to lower-carbon periods without violating deadlines?
- How much simulated carbon reduction is obtained under controlled workloads?
- What performance cost is introduced in waiting time, turnaround time and CPU utilization?
- How does the behavior change as additional CPU scheduling policies are introduced?
- How closely can simulation behavior be reproduced in a controlled virtual-machine environment?

3.3 Project Objectives

- Design an OS-level carbon-aware CPU scheduling mechanism.
- Implement a carbon-aware extension of a classical scheduling policy.
- Use a time-varying carbon-intensity trace as an additional scheduling input.
- Maintain deadline feasibility and avoid indefinite postponement.
- Compare conventional and carbon-aware scheduling using common OS performance metrics.
- Extend the framework to SJF, SRTF, Priority and Round Robin.
- Introduce richer energy and DVFS models after the algorithmic stage.

- Validate the framework inside controlled virtual machines and compare simulation with observed system behavior.

4. Proposed System Architecture

The system is designed as a layered framework. Keeping the layers separate is important because the same experimental infrastructure should eventually support several scheduling algorithms.

1. Workload	2. Base Scheduler	3. Carbon Signal	4. Carbon Decision	5. Execution & Metrics
Processes AT, BT, Deadline	FCFS now SJF / SRTF / Priority / RR later	Time-indexed carbon intensity	Immediate or safe temporal shift	WT, TAT, CPU, Energy, Carbon

The architecture deliberately separates the base scheduler from the carbon-awareness layer. This means that FCFS can be replaced by SJF or Round Robin without redesigning the entire carbon-data and measurement pipeline.

4.1 Inputs

- Process workload: process ID, arrival time, burst time and deadline.
- Carbon trace: time-indexed carbon-intensity values in gCO_2 /kWh.
- Carbon threshold: the value used to classify a period as relatively high-carbon.
- CPU power model: currently a constant 50 W assumption.
- Scheduler policy: FCFS in the current implementation.

4.2 Outputs

- Process start and completion times.
- Waiting time and turnaround time.
- CPU utilization.
- Total modeled energy consumption.
- Estimated carbon emissions.
- Deadline status.
- Gantt-style execution timeline.

5. Carbon-Aware FCFS Algorithm

5.1 Conventional FCFS

In conventional FCFS, processes are served according to their arrival order. Once a process begins, it runs until its CPU burst is completed because FCFS is non-preemptive. The scheduler does not intentionally wait for a different electricity condition.

5.2 Carbon-Aware Extension

The proposed extension preserves the FCFS process order. The only new decision is whether the current start time is environmentally preferable. When a process reaches the front of the queue, the scheduler checks the carbon intensity at the current simulated time.

- If the current carbon intensity is at or below the threshold, the process starts immediately.
- If the current intensity is above the threshold, the scheduler searches forward for a lower-carbon time.
- The candidate start time must allow the complete CPU burst to finish by the process deadline.
- The earliest feasible lower-carbon start is selected.
- If no feasible lower-carbon start exists, the process is not postponed and executes immediately.

5.3 Example of the Decision

Suppose a process arrives at time 0, requires 3 seconds of CPU time and has a deadline of time 10. If carbon intensity at time 0 is high but the trace shows a cleaner period beginning at time 4, the scheduler can postpone the process from time 0 to time 4 because execution from 4 to 7 still completes before the deadline. The scheduler therefore trades four units of simulated waiting for a cleaner execution period.

5.4 Pseudocode

```
CARBON_AWARE_FCFS(processes, carbon_trace):

    arrange processes in FCFS order
    current_time = 0

    for each process P:
        if current_time < P.arrival:
            current_time = P.arrival

        if carbon(current_time) > THRESHOLD:
            find earliest time T such that:
                T >= current_time
                carbon(T) <= THRESHOLD
                T + P.burst <= P.deadline

            if such T exists:
                current_time = T

        P.start = current_time
        P.finish = current_time + P.burst
        P.waiting = P.start - P.arrival
        P.turnaround = P.finish - P.arrival

        current_time = P.finish

    calculate CPU utilization
    calculate energy
    calculate carbon emissions
```

6. Energy and Carbon Accounting

6.1 Current CPU Power Model

The first implementation uses a constant CPU power assumption of 50 W. This is a controlled simulation assumption. It is useful because it isolates the effect of execution timing: the amount of work is unchanged, so the modeled energy remains tied to the total burst time.

6.2 Energy Calculation

For an execution interval of Δt seconds:

Energy (J) = Power (W) × Execution Time (s)

For example, 50 W for 10 seconds gives 500 J of modeled energy.

6.3 Carbon Calculation

Carbon is calculated using the carbon intensity associated with each simulated second. Energy for each second is converted from joules to kWh and multiplied by the carbon intensity for that second.

Carbon (gCO_2) = Σ [Power / 3,600,000 × CarbonIntensity(t)]

This approach means that executing the same workload during a low-carbon period produces a lower estimated carbon value than executing it during a high-carbon period, even when the modeled energy is identical.

IMPORTANT MODELING NOTE

The current carbon values are estimated simulation outputs. They are not direct measurements of emissions from a physical CPU or electricity meter. The VM and later hardware-oriented stages are intended to improve realism.

7. Experimental Methodology

7.1 Experimental Philosophy

The experiments are paired comparisons. Conventional FCFS and carbon-aware FCFS receive the same workload and use the same carbon trace. This is important because changing the workload between algorithms would make it difficult to determine whether a difference came from the scheduler or from the input.

7.2 Experimental Configuration

Parameter	Current configuration
Base policy	FCFS
Carbon-aware policy	FCFS with temporal shifting
Implementation language	C
Execution model	Discrete-event simulation
Carbon trace	Synthetic time-varying trace
Carbon threshold	500 gCO_2 /kWh
CPU power	50 W constant
Workload count	5
Process attributes	PID, arrival time, burst time, deadline
Primary metrics	Carbon, energy, CPU utilization, waiting time, turnaround time

7.3 Workload Design

The five workload configurations contain different numbers and combinations of processes. They were designed to create varied execution lengths and deadlines so that the carbon-aware scheduler encounters different opportunities for temporal shifting. The workload files are kept separate so that experiments can be repeated and later reused for SJF, SRTF, Priority and Round Robin.

7.4 Evaluation Metrics

Metric	Meaning
Average Waiting Time	Measures how long processes remain waiting before receiving CPU service.
Average Turnaround Time	Measures elapsed time from process arrival until completion.
CPU Utilization	Measures the proportion of simulated time during which the CPU is executing work.
Energy	Measures modeled electrical energy under the current CPU power assumption.
Carbon Emissions	Measures estimated carbon associated with the energy at the carbon intensity of each execution period.
Deadline Status	Checks whether each process completes within its specified deadline.

8. Experimental Results

The current figures provide the first complete experimental picture of the proposed scheduler. The most important point is that the carbon-aware scheduler is not expected to improve every traditional CPU-scheduling metric. Its purpose is to introduce an environmental objective, which naturally creates a trade-off with immediate execution.

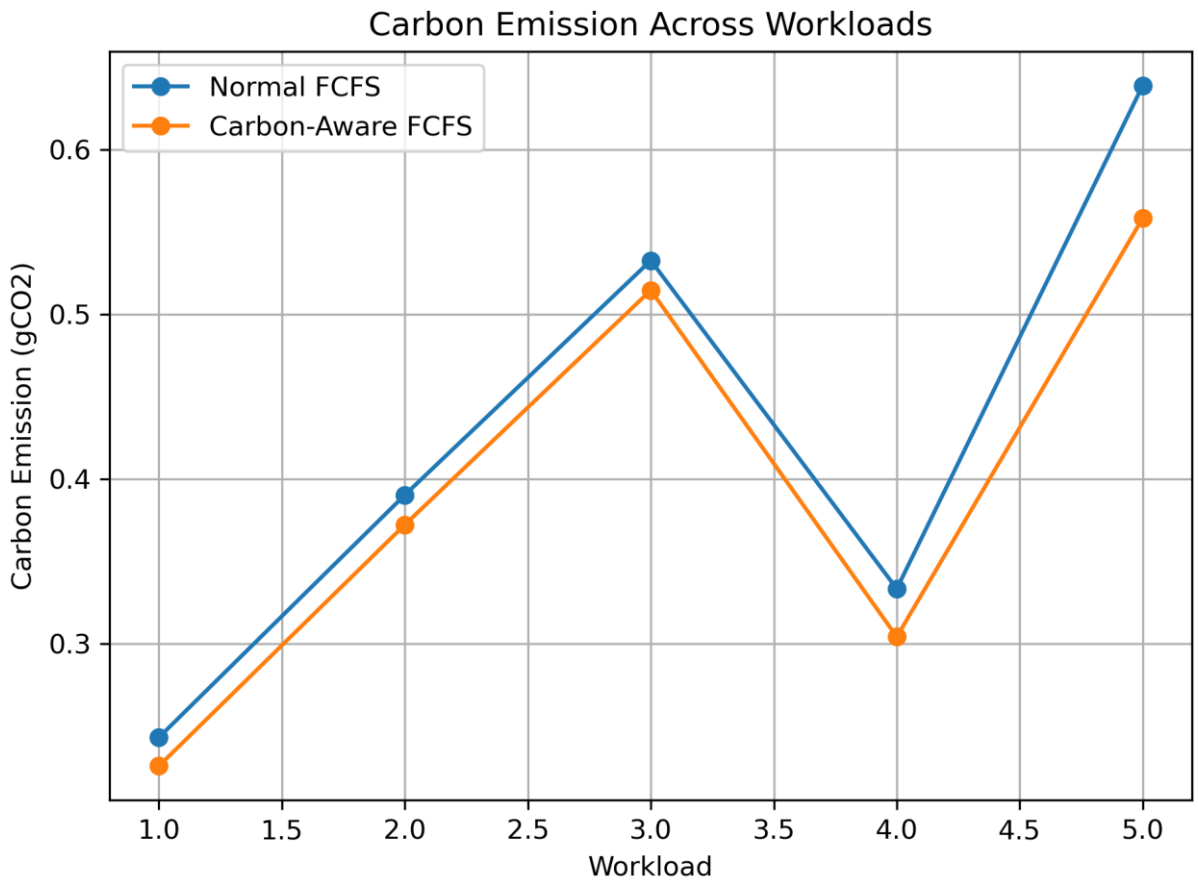


Figure 1. Carbon emissions across workloads. The carbon-aware scheduler remains below conventional FCFS in all five plotted workloads.

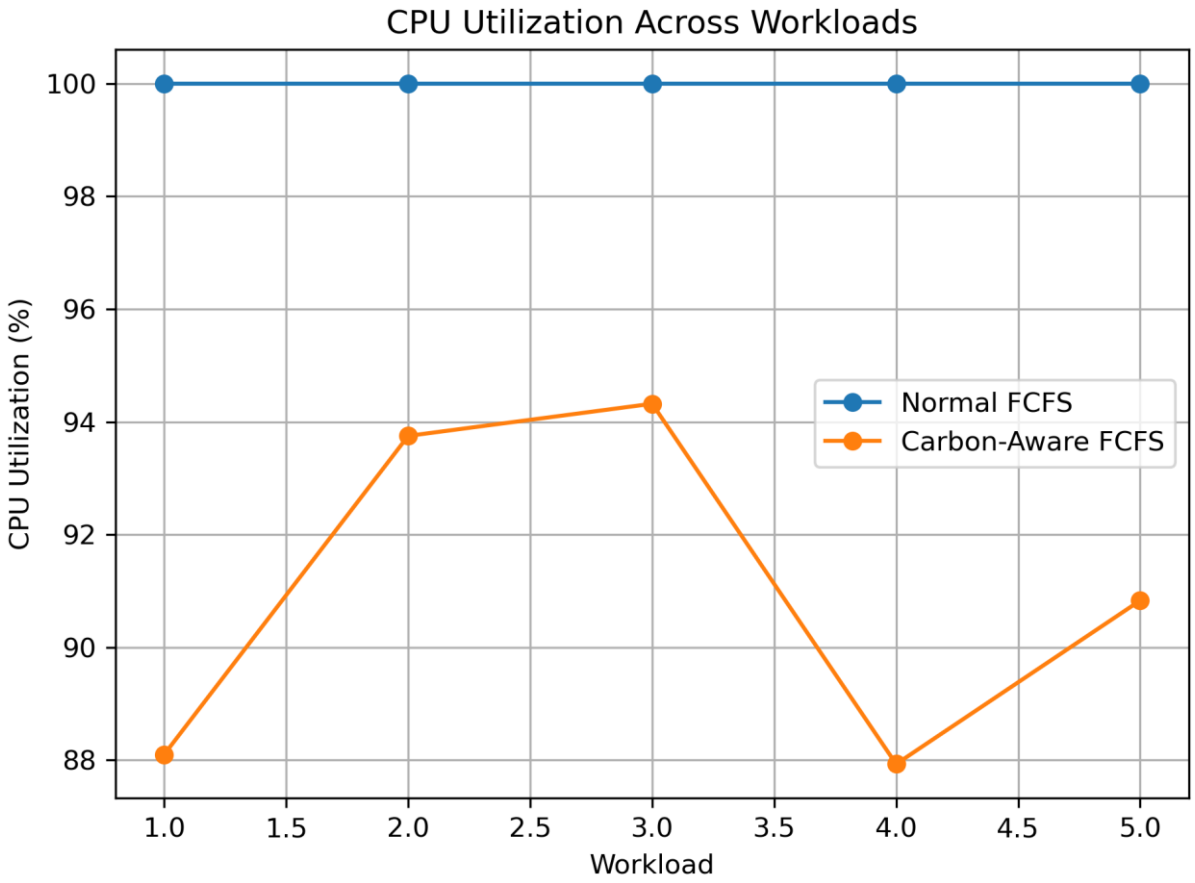


Figure 2. CPU utilization across workloads. Conventional FCFS remains fully busy in the plotted runs, while carbon-aware FCFS introduces intentional idle periods to shift execution.

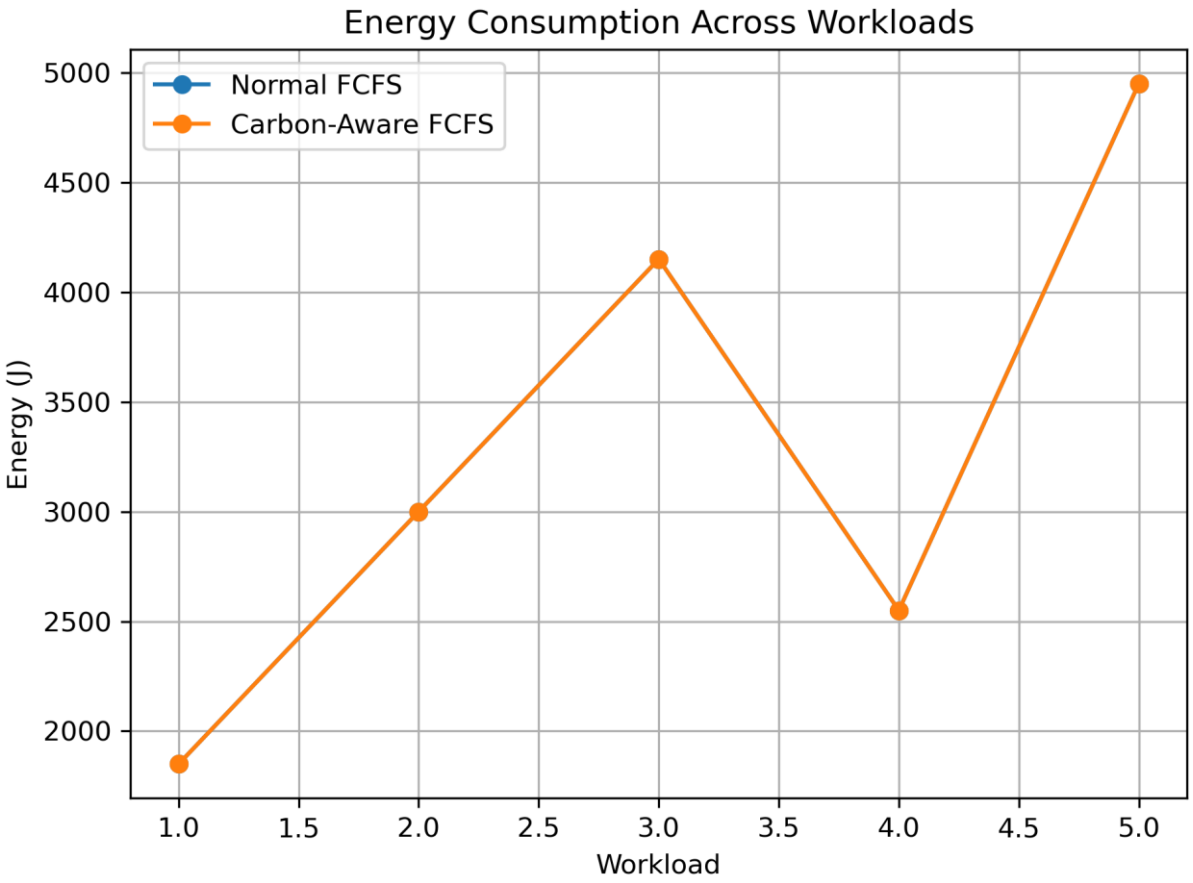


Figure 3. Energy consumption across workloads. The two curves overlap under the current constant-power model because the same total CPU work is executed.

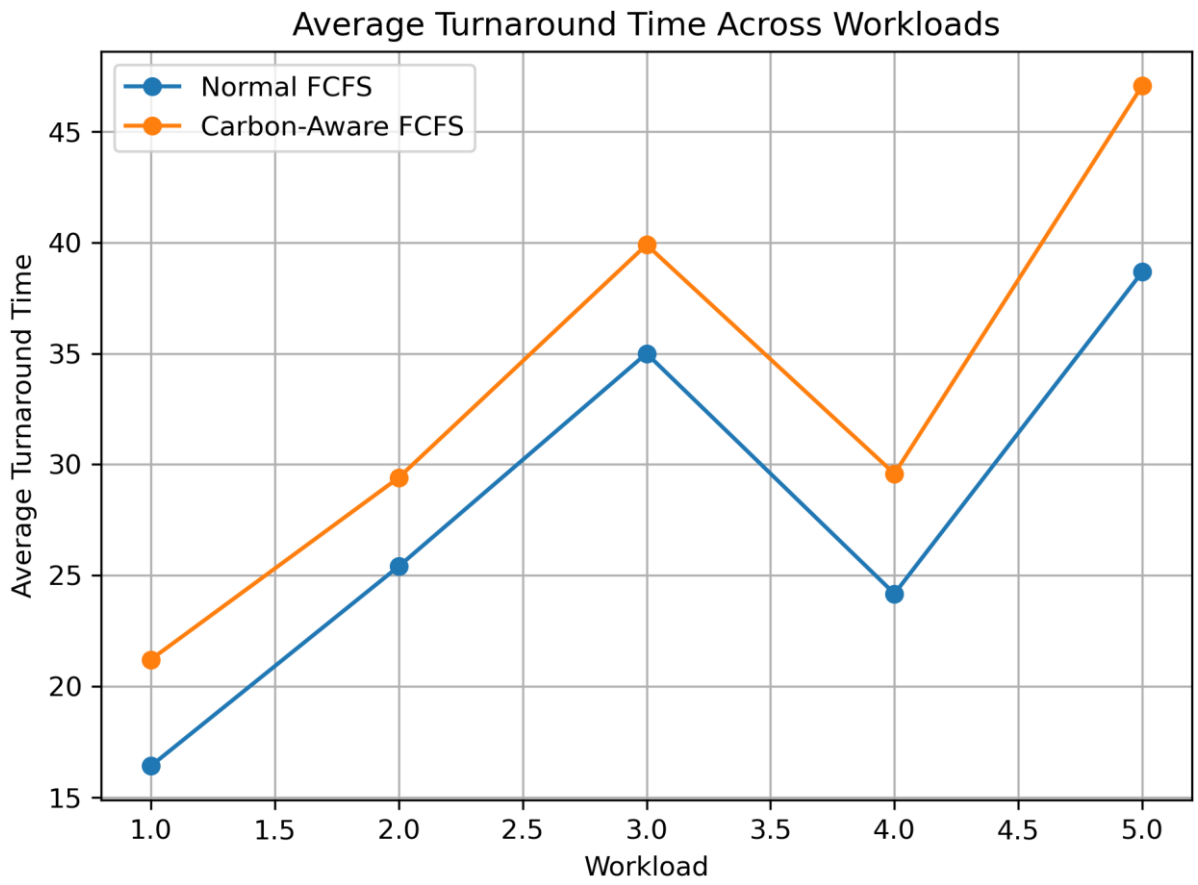


Figure 4. Average turnaround time across workloads. Carbon-aware FCFS generally requires more elapsed time because some processes are postponed.

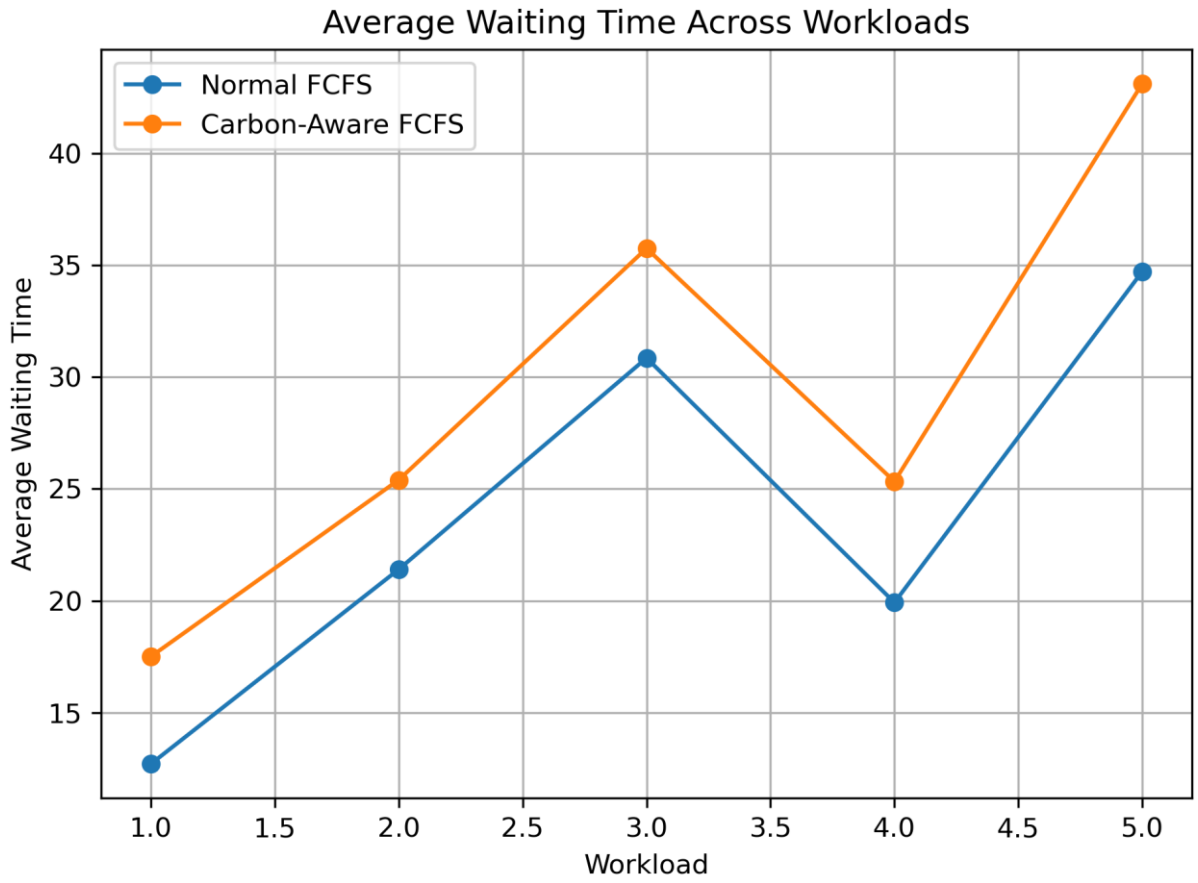


Figure 5. Average waiting time across workloads. The additional waiting reflects the temporal shifting introduced by carbon awareness.

8.1 Carbon Emission Behavior

The carbon-emission figure shows the central outcome of the current experiment. For each of the five workloads, the carbon-aware FCFS curve lies below the conventional FCFS curve. This indicates that the scheduler is successfully exploiting lower-carbon execution periods in the supplied trace.

The reduction is not produced by executing fewer CPU instructions or by reducing the total burst time. Instead, the scheduler changes when the same work is performed. This distinction is central to the research idea and is one reason the carbon figure should be interpreted together with the energy and CPU-utilization figures.

8.2 CPU Utilization Behavior

Conventional FCFS achieves 100% utilization in the plotted workloads because it executes available work continuously. Carbon-aware FCFS shows lower utilization. This is intentional rather than a failure of the scheduler: when a high-carbon period is detected and a feasible cleaner period exists, the scheduler waits instead of consuming electricity immediately.

In a production system, this behavior would need to be controlled carefully. Excessive idle time could damage responsiveness or throughput. Therefore, future versions should introduce maximum-deferral limits, configurable carbon-versus-latency objectives and workload classifications.

8.3 Energy Behavior

The energy figure shows overlapping values for conventional and carbon-aware FCFS. This is expected under the current 50 W constant-power model. Both algorithms execute the same process bursts, so the total modeled CPU energy is effectively the same. Carbon changes because the carbon intensity associated with execution changes.

This result is useful because it demonstrates why carbon awareness should not simply be described as energy saving. A system can consume the same amount of modeled energy and still produce different estimated carbon emissions depending on when the energy is consumed.

8.4 Waiting and Turnaround Behavior

Average waiting time and turnaround time are higher for carbon-aware FCFS in the plotted results. This is a direct consequence of temporal shifting. When the scheduler deliberately postpones execution, the affected process spends more time waiting and its completion time moves later.

These metrics should not be treated as unwanted side effects to be hidden. They are part of the multi-objective scheduling problem. A useful carbon-aware scheduler must expose the relationship between environmental benefit and user-visible performance so that an appropriate trade-off can be selected.

8.5 Supplementary Aggregate Comparisons

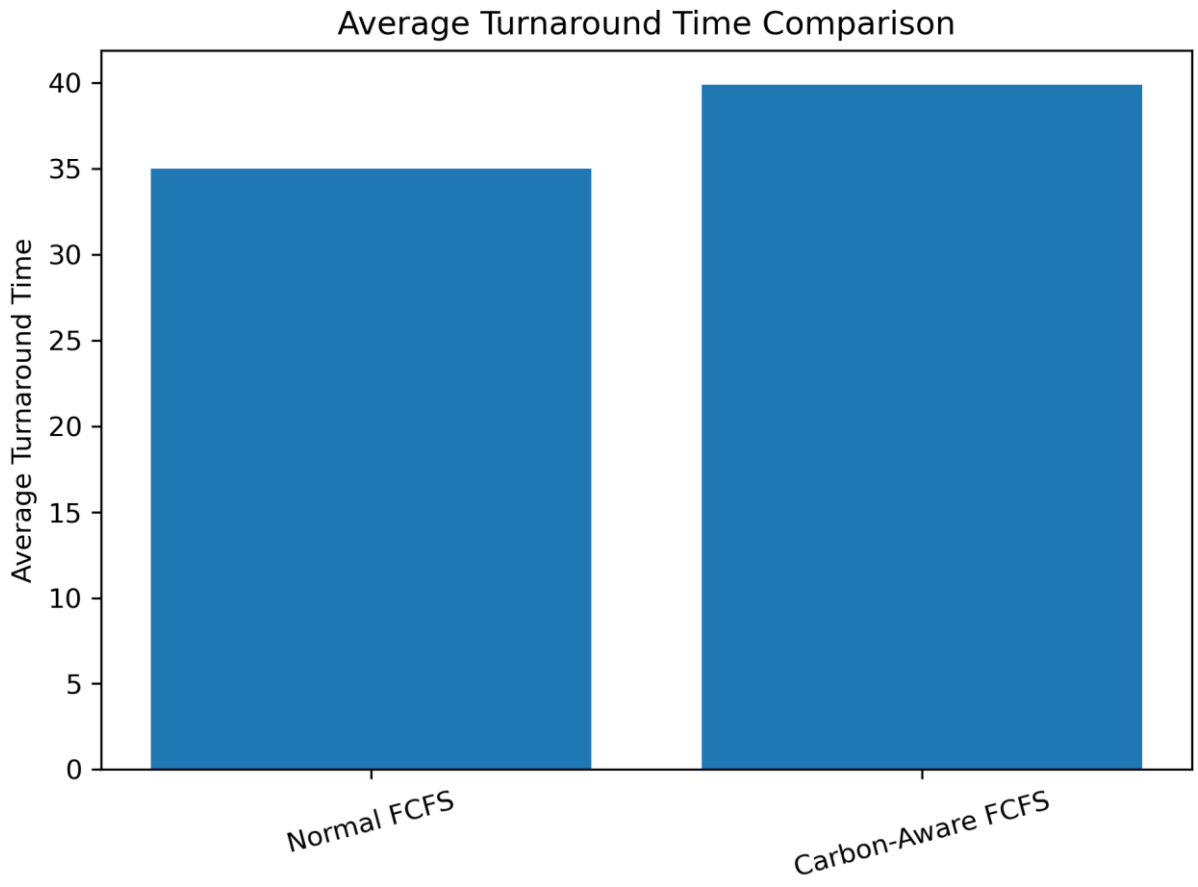


Figure 6. Aggregate average turnaround-time comparison.

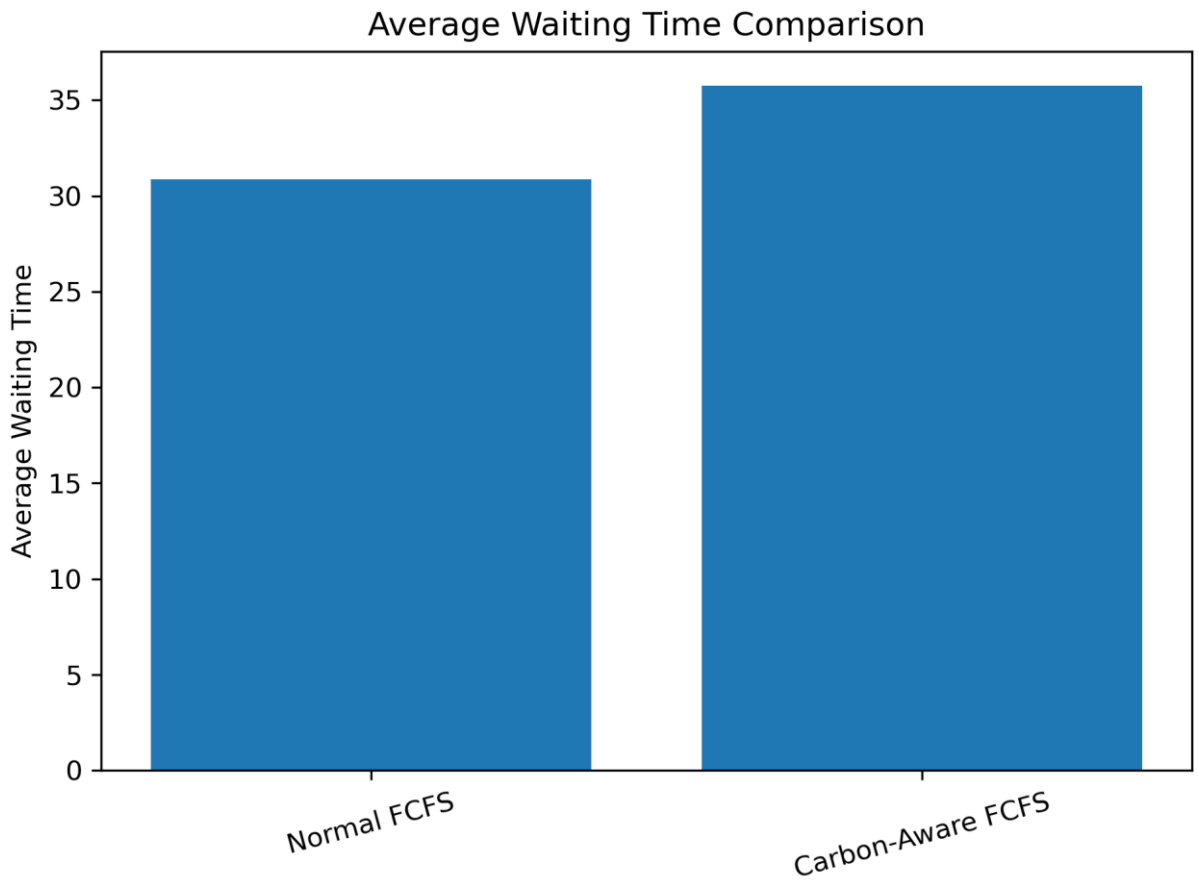


Figure 7. Aggregate average waiting-time comparison.

8.6 Controlled Four-Process Example

A smaller controlled workload was also used to verify the scheduling behavior. Conventional FCFS executed the processes continuously, while carbon-aware FCFS shifted the first process from the initial high-carbon period into a later lower-carbon period while still meeting the deadline.

Metric	Normal FCFS	Carbon-Aware FCFS
Average Waiting Time	2.75	6.75
Average Turnaround Time	5.50	9.50
CPU Utilization	100%	73.33%
Energy	550 J	550 J
Estimated Carbon	0.074306 gCO_2	0.063194 gCO_2
Deadline status	All met	All met

In this particular controlled test, the estimated carbon reduction is approximately 14.95%. This value belongs only to that four-process experiment and should not be presented as the overall performance of the complete project.

9. Discussion

9.1 What the Experiment Demonstrates

The current experiment demonstrates that a classical CPU scheduling policy can be augmented with a temporal carbon signal without completely redesigning the scheduling model. The base FCFS ordering remains understandable and predictable. The carbon-awareness layer acts at the process-start decision, making the prototype easy to reason about and compare against the baseline.

9.2 The Central Trade-Off

The central trade-off is straightforward: lower carbon can require waiting. Conventional FCFS prioritizes immediate service once a process reaches the front of the queue. Carbon-aware FCFS is willing to sacrifice some immediate utilization and responsiveness when the carbon trace suggests that a cleaner period is available.

This trade-off becomes particularly important for different workload classes. Batch jobs with flexible deadlines may tolerate temporal shifting, while interactive or real-time jobs may not. The final framework should therefore avoid assuming that every process has the same environmental flexibility.

9.3 Why the Current Prototype Is Useful

Although the current system is simplified, it provides an important experimental baseline. A controlled simulation makes it possible to isolate the scheduling logic before introducing the additional complexity of Linux scheduling behavior, virtualization overhead, multicore placement and hardware power measurement.

10. Limitations of the Current Study

- Carbon intensity is currently represented by a synthetic trace rather than a live external grid signal.
- The CPU uses a constant 50 W model, so frequency-dependent power behavior is not yet represented.
- The scheduler currently models a single CPU execution stream rather than a multicore system.
- The carbon decision uses a fixed threshold rather than a dynamic optimization objective.
- Only FCFS has been implemented and evaluated in the current research stage.
- Virtual-machine validation has not yet been performed.
- Context-switch overhead, cache effects, thermal behavior and migration overhead are not currently modeled.
- Real-world carbon claims cannot be made from the present simulation alone.

SCOPE BOUNDARY

The present results should be described as estimated simulation results under the stated model. The project will become a systems-validation study only after the planned VM and, if possible, real-machine measurements are completed.

11. Planned Multi-Algorithm and VM Framework

The next stage is to expand the project from a single carbon-aware FCFS implementation into a common scheduling framework. The key design decision is that every algorithm should use the same workload files, carbon trace, metric definitions and result format. This makes comparisons meaningful.

Algorithm	Base scheduling rule	Carbon-aware extension	Status
FCFS	Arrival order	Safe temporal shifting	Current
SJF	Shortest burst	Carbon-aware selection among candidates	Planned
SRTF	Shortest remaining burst	Dynamic carbon-aware decisions	Planned
Priority	Priority value	Priority + carbon feasibility	Planned
Round Robin	Time quantum	Carbon-aware execution/deferral policy	Planned

11.1 Common Experimental Harness

The experiment harness will execute every baseline and carbon-aware algorithm using identical workloads. It will automatically collect results, calculate carbon reduction, calculate performance differences and generate the corresponding figures. This avoids manually changing the test environment for every scheduler.

11.2 Virtual-Machine Validation

After the algorithmic comparison is stable, the project will move into a controlled virtual-machine stage. The VM provides a repeatable Linux environment in which the behavior of the scheduler can be examined under more realistic process execution. The same workload schema and carbon trace can be replayed so that simulation and VM results can be compared.

The VM stage should record the VM configuration, vCPU count, CPU model presented to the guest, workload characteristics, scheduler configuration and measurement method. Where power or energy telemetry is available, it should be recorded separately from the simulation's assumed 50 W model.

12. Reproducibility and Experimental Discipline

A research-quality scheduling project needs more than source code. Each experiment should be reproducible from its input workload, carbon trace, scheduler version and execution command. Results should be stored in machine-readable form before graphs are generated.

- Keep each workload as a separate input file.
- Use the same carbon trace for paired baseline/carbon-aware comparisons.
- Record the scheduler configuration and threshold for every experiment.
- Store raw console output and parsed CSV results.
- Generate figures from the CSV rather than manually entering values.
- Keep simulation results separate from VM measurements.
- Record any changes to the algorithm before rerunning the full benchmark set.

12.1 Suggested Project Structure

```
carbon_aware_scheduler/
│
├── algorithms/
│   ├── fcfs.c
```

```
│   ├── carbon_fcfs.c
│   ├── sjf.c
│   ├── srtf.c
│   ├── priority.c
│   └── round_robin.c
│
├── data/
│   ├── workload1.txt
│   ├── workload2.txt
│   ├── workload3.txt
│   └── ...
│
├── carbon.txt
├── results/
├── graphs/
├── scripts/
└── docs/
```

13. Future Research Directions

13.1 Better Carbon Decision Models

The fixed threshold is intentionally simple. Future versions can use a continuous carbon score, deadline slack, expected future carbon intensity and the cost of delaying a process. This would allow the scheduler to make more nuanced decisions than simply classifying periods as green or non-green.

13.2 DVFS-Aware Scheduling

The original research direction includes CPU frequency as a possible scheduling input. A DVFS-aware extension can model the relationship between frequency, execution time and power. This would allow the project to study both dimensions together: reducing energy through frequency decisions and reducing carbon through temporal decisions.

13.3 Fairness and Responsiveness

Carbon-aware scheduling must eventually consider who experiences the delay. A scheduler that repeatedly postpones particular processes could create unfairness even if the system-wide carbon value improves. Future experiments should therefore include per-process response time, maximum waiting time and fairness measures.

13.4 Realistic Carbon Data

A later experiment should replay a documented real carbon-intensity dataset or a realistic trace rather than relying only on synthetic values. The trace should have a clear time resolution and geographic interpretation so that carbon calculations are reproducible.

14. Conclusion

This project presents a research-oriented approach to bringing carbon awareness into operating-system CPU scheduling. The first implementation extends FCFS with a temporal carbon-awareness layer that can postpone execution when a lower-carbon and deadline-feasible period is available. The current C simulation demonstrates the intended behavior across five workload configurations: simulated carbon emissions decrease, while waiting time and turnaround time increase because execution is deliberately shifted.

The unchanged energy results are equally important. They show that carbon-aware scheduling is not simply another name for energy-efficient scheduling. Under a constant-power model, the same amount of energy can have different estimated carbon consequences depending on when it is consumed.

The project is deliberately structured as a progression rather than a one-off FCFS experiment. The next stage will introduce SJF, SRTF, Priority and Round Robin while preserving a common workload and measurement framework. The subsequent stage will enrich the power model, introduce DVFS and move the evaluation into controlled virtual machines. This progression will allow the final study to move from a simple proof of concept toward a more complete OS scheduling evaluation.

Ultimately, the value of the proposed framework lies in treating environmental impact as one of the dimensions of CPU scheduling while still measuring the traditional operating-system objectives that users and systems depend on. The project therefore aims not merely to reduce a carbon number, but to characterize the relationship between carbon reduction, system performance, deadline feasibility and scheduling behavior.

15. Selected References

1. A. Radovanović et al., “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, 2023.

2. “A Green Cloud-Based Framework for Energy-Efficient Task Scheduling Using Carbon Intensity Data for Heterogeneous Cloud Servers,” IEEE Xplore, 2025.

3. “Green Scheduling for Cloud Data Centers Using Renewable Resources,” IEEE Conference Publication.

4. “A Carbon-Aware Workload Dispatcher in Cloud Computing Systems,” IEEE Conference Publication, 2023.

5. “Electricity and Carbon-Aware Task Scheduling in Geo-Distributed Internet Data Centers,” IEEE Conference Publication.

6. “Does Energy-Efficiency Mean Carbon-Awareness? A Critical Evaluation of Sustainable Serverless Edge Scheduling,” IEEE/ACM UCC, 2025.

7. G. Perin et al., “EASE: Energy-Aware Job Scheduling for Vehicular Edge Networks with Renewable Energy Resources.”

8. S. Zhang et al., “Sustainable AIGC Workload Scheduling of Geo-Distributed Data Centers: A Multi-Agent Reinforcement Learning Approach,” 2023.

9. Z. Song et al., “A Carbon-Aware Framework for Energy-Efficient Data Acquisition and Task Offloading in Sustainable AIoT Ecosystems,” IEEE Internet of Things Journal, 2024.

10. D. Yan, M.-Y. Chow, Y. Chen, “Low-Carbon Operation of Data Centers with Joint Workload Sharing and Carbon Allowance Trading,” IEEE Transactions on Cloud Computing, 2024.

11. C. Scordino, L. Abeni, J. Lelli, “Energy-Aware Real-Time Scheduling in the Linux Kernel,” ACM SAC, 2018.

12. F. Qiao et al., “Energy-Aware Process Scheduling in Linux,” ACM SIGEnergy Energy Informatics Review, 2025.

13. “Energy-Aware Non-Preemptive Task Scheduling with Deadline Constraint in DVFS-Enabled Heterogeneous Clusters.”

14. “Thermal-Aware Energy-Efficient Task Scheduling for DVFS-Enabled Data Centers.”

15. “Energy-Aware Scheduling of Real-Time Workflow Applications in Clouds Utilizing DVFS and Approximate Computations.”

FINAL NOTE

The current FCFS results form the first experimental chapter of the project. Future results should be added using the same metric definitions and experimental discipline so that the final report can present a coherent multi-algorithm research study.